

Informações:

- Esta aplicação extrai dados das tabelas **FatoCompra** e **FatoVenda** e realiza correções conforme necessário.
- **Funcionalidades:**
 -  **Correção do mês atual:** O mês é processado por completo, dividido em períodos de 7 dias, começando do último dia até o primeiro. Se houver divergências em algum período, o script percorre os dias desse intervalo para identificar exatamente em quais dias as inconsistências ocorrem e corrigi-las individualmente.
 -  **Carga diária:** Realiza a carga de dados do dia anterior (Dia -1).

Arquivos Files

Módulos (arquivos TXT):

Separamos alguns scripts em módulos para facilitar a manutenção do código. Esses módulos incluem:

- Variáveis de configuração da aplicação principal (Main).
- Caminhos ("path") de conexões.
- Sub-rotinas responsáveis pela geração de arquivos QVD.
- Sub-rotinas utilizadas para a extração de dados

Módulos são arquivos que contêm código reutilizável e podem ser importados por diferentes aplicações

```
/*
    Esse código busca as variáveis de configuração Main(Variáveis de controle de datas e conversões)
    Também possui a variável de controle de duração da carga (vStartLoad)
*/
$(Include=[lib://Services ETL:DataFiles/VAR_MAIN_PTBR.txt])

/*
    Esse código busca as variáveis de conexão
    Variáveis:
        caminho da extração: vExtractedPath = 'lib://Services ETL:DataFiles';
        conexão com o Banco de Dados: vConnectionC5 = 'Desenvolvimento:Consinco';
        conexão: LIB CONNECT TO '${vConnectionC5}';
*/
$(Include=[lib://Services ETL:DataFiles/VAR_PATH_CONNECTION_C5.txt])

/*
    Esse código busca as SubRotinas de criação de arquivos e Registros de tempos de carga
    SubRotina de criação QVD:
        RecordQVD(pTable) > recebe como parâmetro o nome da tabela para criação do arquivo

    SubRotina de criação CSV:
        RecordCSV(pTable,vFile) > recebe como parâmetro o nome da tabela (pTable) que será criado o QVD,
        (vFile) recebe uma nomenclatura que o nome do arquivo QVD irá receber

    SubRotina para registrar o tempo de Leitura das extrações
    Essa SubRotina é chamada pelas SubRotinas de criação de arquivos (RecordQVD, RecordCSV)
    ReadingTime(pTable,pInitTime,pFinalTime) > Recebe como parametro o nome da tabela que foi gerada (pTable)
    O tempo que começou a carga e o tempo que finalizou (pInitTime,pFinalTime)

    SubRotina para armazenar os registros de leitura e extrações em uma tabela que fica na memória do qlick
    Essa SubRotina é chamada pela SubRotina(ReadingTime).
    ReadingTime(pTable,pInitTime,pFinalTime) > Recebe como parametro o nome da tabela que foi gerada (pTable)
    e o tempo de duração da carga (pInitTime,pFinalTime)
*/
$(Include=[lib://Services ETL:DataFiles/SUB_RECORDQVD.txt])
```

Carga Completa

Variaveis de controle de carga:

vYesterday - recebe a data de ontem

vDay – recebe o "dia" da data de ontem

vInitMonth – recebe um valor numérico que vai determinar qual mês a carga irá iniciar. Se o mês for igual a (0), a carga irá iniciar do mês atual, se for igual a (- 2) a carga irá iniciar a partir de 2 meses atrás.

If vDay= 7 then

vInitMonth = -1;

end if;

Este **IF** verifica se o dia do mês atual é 7.

Se for, **vInitMonth** recebe o valor -1, indicando que a carga deve iniciar a partir do mês anterior. Essa verificação garante que, todo dia 7, o sistema retorne ao mês anterior para realizar a correção completa.

Call RecoverMonths(vInitMonth,0,'Parametro',vYesterday) : Chama a Subrotina

RecoverMonths Passando os parâmetros:

(Valor numérico inicial mês, Valor numérico final mês,
'Quais tabelas vão carregar', data de ontem)

3º Parâmetro da Sub-rotina RecoverMonths:

1 – se quiser fazer carga somente na fato venda

2 – se quiser fazer carga somente na fato compra

3 – se a carga for em ambas

```
LET vYesterday = Today()-1;  
LET vDay = Day(vYesterday);  
LET vInitMonth = 0;
```

// Verifica Se é dia 7 e aplica a carga de correção mensal + a diária.

```
If vDay = 7 then  
    vInitMonth = -1;  
end if;
```

*/**

Legenda dos Parâmetros:

1º Mês inicial (sempre preencher com valores negativos).

2º mês final, qual o mês que a carga deve parar, sendo 0 o mês atual;

3º Qual tabela carregará, 1 = venda, 2 = Compra, 3 = ambas.

**/*

```
CALL RecoverMonths(vInitMonth,0,'3',vYesterday);
```

```
LET vFullload = NOW();
```

```
CALL ReadingTime('TOTAL',vStartLoad,vFullload);
```

```
Exit Script;
```

Sub-Rotina Recover Months

Sub RecoverMonths:

Essa Sub-rotina vai definir quantas tabelas iremos fazer a correção e a partir de qual mês a carga irá iniciar.

A Sub Recebe 4 Parâmetros:

Parâmetro1: recebe um valor numérico que vai determinar qual mês a carga irá iniciar

Parâmetro2: recebe um valor numérico que vai determinar qual mês a carga irá finalizar

Parâmetro3: Pode receber os valores 1, 2 ou 3:

Parâmetro4: Recebe a data de ontem que será utilizada como referência para a correção do mês

Variáveis:

vDate – Recebe uma data **Today()-1**, com o mês definido pelo **AddMonths()**.

Se o valor de **pStartMonth** for -1, então **vDate** assume o dia de ontem e o mês será referente ao mês anterior.

Temos um **For** para percorrer o mês início até o mês final, criando uma variável de controle de data que é passada como parâmetro quando chamamos a Sub-Rotina **CorrMonth**.

IF - para controle de qual tabela iremos carregar

Chamamos a Sub-Rotina **CorrMonth('nome_da_tabela', vDate)**

```
/*  
    Essa sub é destinada ao controle da carga.  
    Opções para Saber em qual FATO Será gerada a carga e em qual mês  
*/  
SUB RecoverMonths(pStartMonth,pEndMonth,pWhichLoad,pDateRef)  
    for vIndexMonth = pStartMonth to pEndMonth  
        LET vDate = AddMonths(pDateRef,vIndexMonth);  
        If pWhichLoad = 1 or pWhichLoad = 3 then  
            CALL CorrMonth('FatoVenda',vDate);  
        end if;  
        If pWhichLoad = 2 or pWhichLoad = 3 then  
            CALL CorrMonth('FatoCompra',vDate);  
        end if;  
    next;  
END SUB;
```

Sub-Rotina CorrMonth

Sub CorrMonth:

Sub-Rotina tem o objetivo de verificar se há divergências no mês, no número de linhas ente a consulta ao DW e ao arquivo CSV que temos carregado dentro do Qlick. Se for encontrado divergência, chama a Sub-rotina CorrPeriod que divide o mês em períodos e verifica em qual período do mês se encontra a divergência.

A Sub Recebe 2 Parâmetros:

\$(pSubFato) - É o nome da tabela que foi passada como parâmetro na Sub RecoverMonths ('FatoVenda','FatoCompra')

vDateRef - Recebe a data de referência para começar a carga mensal, a partir desta data, pegamos o primeiro dia do mês e o último.

Variáveis:

vDateOneMonthAgo – recebe a data que foi passada como parâmetro (vDateRef)

vFirstDayOneMonthAgo – Recebe o primeiro dia do mês da data que foi passada como referência

vLastDayOneMonthAgo - Recebe o último dia do mês da data que foi passada como referência

vContSelect - Variável que vai receber a quantidade de linhas do select. Rebece um valor padrão de (0).

vCountLinesCsv - Variável que vai receber a quantidade de linhas do arquivo CSV existente. Rebece um valor padrão de (0).

vTotalDivergence - Recebe o valor da divergência das quantidades de linhas (vContSelect - vCountLinesCsv);

```
//Essa sub vai verificar se contem divergencias no mês - entre as informações do (DW e ARQUIVOS CSV)
```

```
SUB CorrMonth(pSubFato,pDateRef)
TRACE CORRIGINDO O ARQUIVO >>> $(pSubFato);
LET vContSelect = 0;
LET vCountLinesCsv = 0;
Let vDateOneMonthAgo = pDateRef;
LET vFirstDayOneMonthAgo = MonthStart(vDateOneMonthAgo);
LET vLastDayOneMonthAgo = MonthEnd(vDateOneMonthAgo);

TRACE # $(vDateOneMonthAgo) #;

CALL $(pSubFato)CountLinesSelect('TempCountSelect',vFirstDayOneMonthAgo,vLastDayOneMonthAgo);
CALL $(pSubFato)CountLinesCsv('TempTableCsv',vFirstDayOneMonthAgo,vLastDayOneMonthAgo);

LET vTotalDivergence = (vContSelect - vCountLinesCsv);
LET vCtrlDivergenceMonth = vTotalDivergence;

//Exclui as tabelas que estavam armazenando os totais de linhas
Trace $(vFirstDayOneMonthAgo) <> $(vLastDayOneMonthAgo) Total Divergencias = $(vTotalDivergence) ($(vContSelect) - $(vCountLinesCsv)) ;

If vTotalDivergence <> 0 then
    CALL CorrPeriod(pSubFato,vFirstDayOneMonthAgo,vLastDayOneMonthAgo,vCtrlDivergenceMonth)
End If;

END SUB;
```

Sub-Rotina CorrMonth

Sub CorrMonth:

Sub-Rotina tem o objetivo de verificar se há divergências no mês, no número de linhas ente a consulta ao DW e ao arquivo CSV que temos carregado dentro do Qlick. Se for encontrado divergência, chama a Sub-rotina CorrPeriod que divide o mês em períodos e verifica em qual período do mês se encontra a divergência.

Sub-Rotina \$(pSubFato)CountLinesSelect :

\$(pSubFato)CountLinesSelect('TempCountSelect',vFirstDayOneMonthAgo,vLastDayOneMonthAgo)

Sub-rotina Tem como objetivo contar as linhas do select no Dw.

O parâmetro **\$(pSubFato)** no início da sub é o que vai definir em qual tabela countlines vai chamar

('FatoVenda**CountLinesSelect**', 'FatoCompra**CountLinesSelect**').

Recebe como parâmetros:

'TempCountSelect' - nome da tabela temporária que irá armazenar a quantidade de linhas resultante do select.

vFirstDayOneMonthAgo - primeiro dia do mês

vLastDayOneMonthAgo - último dia do mês

Sub-Rotina \$(pSubFato)CountLinesCsv:

CALL \$(pSubFato)CountLinesCsv('TempTableQvd',vFirstDayOneMonthAgo,vLastDayOneMonthAgo);

Sub-rotina Tem como objetivo contar as linhas do arquivo CSV existente no Qlick.

O parâmetro **\$(pSubFato)** no início da sub é o que vai definir em qual tabela countlines vai chamar

('FatoVenda**CountLinesCsv**', 'FatoCompra**CountLinesCsv**').

Recebe como parâmetros:

'TempTableQvd'- nome da tabela que irá armazenar a quantidade de linhas resultante do arquivo CSV..

vFirstDayOneMonthAgo - primeiro dia do mês

vLastDayOneMonthAgo - último dia do mês

//Essa sub vai vericar se contem divergencias no mês - entre as informações do (DW e ARQUIVOS CSV)

```
SUB CorrMonth(pSubFato,pDateRef)
TRACE CORRIGINDO O ARQUIVO >>> $(pSubFato);
LET vContSelect = 0;
LET vCountLinesCsv = 0;
Let vDateOneMonthAgo = pDateRef;
LET vFirstDayOneMonthAgo = MonthStart(vDateOneMonthAgo);
LET vLastDayOneMonthAgo = MonthEnd(vDateOneMonthAgo);

TRACE # $(vDateOneMonthAgo) #;

CALL $(pSubFato)CountLinesSelect('TempCountSelect',vFirstDayOneMonthAgo,vLastDayOneMonthAgo);
CALL $(pSubFato)CountLinesCsv('TempTableCsv',vFirstDayOneMonthAgo,vLastDayOneMonthAgo);

LET vTotalDivergence = (vContSelect - vCountLinesCsv);
LET vCtrlDivergenceMonth = vTotalDivergence;

//Exclui as tabelas que estavam armazenando os totais de linhas

Trace $(vFirstDayOneMonthAgo) <> $(vLastDayOneMonthAgo) Total Divergencias = $(vTotalDivergence) ($(vContSelect) - $(vCountLinesCsv)) ;

If vTotalDivergence <> 0 then
    CALL CorrPeriod(pSubFato,vFirstDayOneMonthAgo,vLastDayOneMonthAgo,vCtrlDivergenceMonth)
End If;

END SUB;
```

Sub-Rotina CorrMonth

If vTotalDivergence <> 0 then

CALL CorrPeriod(pSubFato,vFirstDayOneMonthAgo,vLastDayOneMonthAgo,vTotalDivergence)

End If;

Este IF verifica se há divergências no mês analisado.

Se houver, a sub-rotina **CorrPeriod** é chamada para percorrer os períodos do mês e identificar quais possuem inconsistências para correção.

```
//Essa sub vai verificar se contem divergencias no mês - entre as informações do (DW e ARQUIVOS CSV)
SUB CorrMonth(pSubFato,pDateRef)
TRACE CORRIGINDO O ARQUIVO >>> $(pSubFato);
LET vContSelect = 0;
LET vCountLinesCsv = 0;
Let vDateOneMonthAgo = pDateRef;
LET vFirstDayOneMonthAgo = MonthStart(vDateOneMonthAgo);
LET vLastDayOneMonthAgo = MonthEnd(vDateOneMonthAgo);

TRACE # $(vDateOneMonthAgo) #;

CALL $(pSubFato)CountLinesSelect('TempCountSelect',vFirstDayOneMonthAgo,vLastDayOneMonthAgo);
CALL $(pSubFato)CountLinesCsv('TempTableCsv',vFirstDayOneMonthAgo,vLastDayOneMonthAgo);

LET vTotalDivergence = (vContSelect - vCountLinesCsv);
LET vCtrlDivergenceMonth = vTotalDivergence;

//Exclui as tabelas que estavam armazenando os totais de linhas

Trace $(vFirstDayOneMonthAgo) <> $(vLastDayOneMonthAgo) Total Divergencias = $(vTotalDivergence) ($(vContSelect) - $(vCountLinesCsv)) ;

If vTotalDivergence <> 0 then
    CALL CorrPeriod(pSubFato,vFirstDayOneMonthAgo,vLastDayOneMonthAgo,vCtrlDivergenceMonth)
End If;

END SUB;
```

Sub-Rotina CountLinesSelect

Sub CountLinesSelect:

Esta sub-rotina conta as linhas do SELECT e armazena o resultado em uma tabela temporária, permitindo que o valor seja acessado posteriormente.

A Sub Recebe 3 Parâmetros:

pTableName - Nome da tabela temporária, para armazenamento da quantidade de linhas.

pFirstDayOneMonthAgo - dia inicial que o select vai começar a contar.

pLastDayOneMonthAgo - dia final que o select vai parar de contar.

Variáveis:

vContSelect – Recebe a quantidade de linhas resultante do select;

Drop Table \$(pTableName) - Após alocarmos a quantidade de linhas em uma variável, deletamos a tabela temporária.

```
//Sub que conta linhas retornadas do select Tabela "FATO_VENDA"  
SUB FatoVendaCountLinesSelect(pTableName,pFirstDayOneMonthAgo,pLastDayOneMonthAgo)  
  $(pTableName):  
  SQL SELECT  
    COUNT(*) AS CountLineSelect  
  FROM CONSINCODW.FATO_VENDA FV  
  WHERE  
    FV.DTAOPERACAO BETWEEN TO_DATE('$(pFirstDayOneMonthAgo)', 'DD/MM/YYYY') AND TO_DATE('$(pLastDayOneMonthAgo)', 'DD/MM/YYYY');  
  vContSelect = Peek('COUNTLINESELECT', 0, 'TempCountSelect');  
  Drop TABLE $(pTableName);  
END SUB;
```


Sub-Rotina CountLinesCsv

Sub CountLinesCsv:

Esta sub-rotina conta as linhas de um arquivo CSV e armazena o resultado em uma tabela temporária, permitindo que o valor seja acessado posteriormente.

A Sub Recebe 3 Parâmetros:

pTableName - Nome da tabela temporária, para armazenamento da quantidade de linhas.

pFirstDayOneMonthAgo - dia inicial que o select vai começar a contar.

pLastDayOneMonthAgo - dia final que o select vai parar de contar.

Variáveis

vYear – Recebe o ano referente a data **pFirstDayOneMonthAgo**

vMonth - Recebe o mês referente a data **pFirstDayOneMonthAgo**

vCountLinesCsv - Recebe a quantidades de linhas do arquivo CSV

Essas variáveis é para controle de qual arquivo iremos abrir

[\$(vExtractedPath)/Consinco_Vendas_\$(vYear)_\$(vMonth).csv]

If FileSize('\$(vExtractedPath)/Consinco_Vendas_\$(vYear)_\$(vMonth).csv') > 0:

Esse IF é para verificar se o arquivo procurado para contar as linhas existe no Qlick, se ele não existir a variável **vCountLinesCsv** mantém o valor zerado.

Drop Table \$(pTableName) - Após alocarmos a quantidade de linhas em uma variável, deletamos a tabela temporária.

```
//Sub que conta Linhas retornadas do arquivo CSV da Tabela "CONSINCO_VENDAS"
SUB FatoVendaCountLinesCsv(pTableName,pFirstDayOneMonthAgo,pLastDayOneMonthAgo)
    LET vYear = Year(pFirstDayOneMonthAgo);
    LET vMonth = Num(Month(pFirstDayOneMonthAgo),00);

    If FileSize('$(vExtractedPath)/Consinco_Vendas_$(vYear)_$(vMonth).csv') > 0 then
        $(pTableName):
        LOAD
            RowNo() as Lines
        FROM
            [$(vExtractedPath)/Consinco_Vendas_$(vYear)_$(vMonth).csv]
            (txt, utf8, embedded labels, delimiter is '|')
        Where DATE(DTAOPERACAO,'DD/MM/YYYY') >= '$(pFirstDayOneMonthAgo)' and DATE(DTAOPERACAO,'DD/MM/YYYY') <= '$(pLastDayOneMonthAgo)';
        vCountLinesCsv = NoOfRows('TempTableCsv');
        Drop TABLE $(pTableName);
    End if;
END SUB;
```

SubRotina CorrPeriod

Sub CorrPeriod:

Esta sub-rotina verifica se há divergências dentro dos períodos de um mês, dividindo-o em períodos de 7 dias. Ao identificar um período com divergência, chama a sub-rotina de correção de dia (CorrDay), que analisa cada dia do período e realiza as correções necessárias.

A Sub Recebe 4 Parâmetros:

pSubFato - É o nome da tabela que foi passada como parâmetro na Sub RecoverMonths('FatoVenda','FatoCompra').

pFirstDayOneMonthAgo - Recebe o primeiro dia do mês.

pLastDayOneMonthAgo - Recebe o primeiro dia do mês.

pCtrlDivergence – Recebe o total de divergência do mês.

Variáveis:

vStep = -7; - Valor do período que o mês será dividido (será dividido em 7 dias)

vCtrlDivergencePeriod - Recebe o valor do parâmetro (**pCtrlDivergence**) que é o total de divergências do mês completo para controle de quando encerrar a correção do período.

vContSelect – Recebe a quantidade de linhas, após a chamada da função **\$(pSubFato)CountLinesSelect**

vCountLinesCsv - Recebe a quantidade de linhas, após a chamada da função **\$(pSubFato)CountLinesCsv**

VDivergencePeriod - Recebe o valor da divergência das quantidades de linhas (**vContSelect - vCountLinesCsv**);

```
//Essa sub vai verificar se contem divergencias nos períodos do Mês: a cada (7 dias) Começando do ultimo periodo do mês
Sub CorrPeriod(pSubFato,pFirstDayOneMonthAgo,pLastDayOneMonthAgo,pCtrlDivergence)
    LET vStep = -7;
    SET fCalcDivgCtrl = ($1 - ( $2 - $3));
    LET vCtrlDivergencePeriod = pCtrlDivergence;
    for vIndexPeriod = pLastDayOneMonthAgo to pFirstDayOneMonthAgo step vStep
        LET vLastDayPeriod = date(vIndexPeriod);
        LET vFirstDayPeriod = date(vIndexPeriod+(vStep+1));

        If day(vFirstDayPeriod) > day(vLastDayPeriod) then
            vFirstDayPeriod = MonthStart(vLastDayPeriod);
        end if;

        CALL $(pSubFato)CountLinesSelect('TempCountSelect',vFirstDayPeriod,vLastDayPeriod);
        CALL $(pSubFato)CountLinesCsv('TempTableCsv',vFirstDayPeriod,vLastDayPeriod);

        LET vContSelectPeriod = Peek('COUNTLINESELECT', 0, 'TempCountSelect');
        LET vCountLinesCsvPeriod = NoOfRows('TempTableCsv');
        LET vDivergencePeriod = (vContSelectPeriod - vCountLinesCsvPeriod);

        Trace Entre $(vFirstDayPeriod) e $(vLastDayPeriod). Divergência de: $(vDivergencePeriod);

        CALL CleanTemps;

        vCtrlDivergencePeriod = $(fCalcDivgCtrl(vCtrlDivergencePeriod,vContSelectPeriod,vCountLinesCsvPeriod));

    If vDivergencePeriod <> 0 then
        CALL CorrDay(pSubFato,vFirstDayPeriod,vLastDayPeriod,vDivergencePeriod);
    End If;

    Trace PERÍODO -> Total = $(vTotalDivergence). Controlador = $(vCtrlDivergencePeriod). # ( $(vContSelectPeriod) - $(vCountLinesCsvPeriod) ) #;

    If vCtrlDivergencePeriod = 0 then
        Exit for;
    end if;
next;
END SUB;
```

SubRotina CorrPeriod

Função fCalcDivgCtrl = (\$1 - (\$2 - \$3)):

Esta função atualiza o valor da divergência a cada loop em que uma inconsistência é identificada e corrigida. Seu objetivo é reduzir gradualmente as divergências até que o valor seja zerado.

Parâmetros: (vCtrlDivergencePeriod,vContSelect,vCountLinesCsv)

for vIndexPeriod = pLastDayOneMonthAgo to pFirstDayOneMonthAgo vStep:

Loop para percorrer os períodos do mês, começando do último dia do mês, de 7 em 7.

vLastDayPeriod = date(vIndexPeriod); - Recebe o último dia do período percorrido.

FirstDayPeriod = date(vIndexPeriod+(vStep+1)); - Recebe último dia do período percorrido – 7, ou seja. Se o último dia for igual a 31, **FirstDayPeriod** vai receber 24. Formando o primeiro período a ser verificado do dia 31 até 24.

If day(vFirstDayPeriod) > day(vLastDayPeriod) then

vFirstDayPeriod = MonthStart(vLastDayPeriod);

end if;

Este **IF** garante que o primeiro dia nunca seja maior que o último dia do período.

Por exemplo, se o último dia for 6, o cálculo (**último dia - 7**) resultaria em um primeiro dia pertencente ao mês anterior. Para evitar essa situação, essa verificação assegura que o primeiro dia nunca seja anterior ao primeiro dia do mês atual.

CALL\$

(pSubFato)CountLinesSelect('TempCountSelect',vFirstDayPeriod,vLastDayPeriod);

Chamamos a subrotina **CountLinesSelect** para contar a quantidade de linhas do dw passando os períodos como parametros

CALL \$(pSubFato)CountLinesCsv('TempTableCsv',vFirstDayPeriod,vLastDayPeriod);

Chamamos a subrotina **CountLinesCsv** para contar a quantidade de linhas do arquivo CSV passando os períodos como parâmetros

IF vDivergencePeriod <> 0 then

CALL CorrDay(pSubFato,vFirstDayPeriod,vLastDayPeriod,vDivergencePeriod);

End If;

Este IF verifica se há divergências no período analisado.

Se houver, a sub-rotina **CorrDay** é chamada para percorrer os dias do período e identificar quais possuem inconsistências para correção.

If vCtrlDivergencePeriod = 0;

Exit For;

End If;

Quando a variável que armazena o total de divergências (**vCtrlDivergencePeriod**) atingir zero, indicando que todas as inconsistências foram corrigidas, o loop é interrompido.

SubRotina CorrDay

Sub CorrDay:

Esta sub-rotina verifica se há divergências nos dias de um período passado. Caso encontre alguma divergência, chama a sub-rotina de correção de carga (CALL \$(pSubFato)(vDateRef)), que extrai os dados do dia informado como parâmetro e corrige o arquivo CSV existente.

A Sub Recebe 4 Parâmetros:

pSubFato - É o nome da tabela que foi passada como parâmetro na Sub RecoverMonths ('FatoVenda','FatoCompra') ela que auxilia em qual tabela estamos fazendo a correção

pFirstDayPeriod – Recebe o primeiro dia do período

pLastDayPeriod – Recebe o último dia do período

pCtrlDivergencePeriod – Recebe o total de divergência do período passado

Variaveis:

vCtrlDivergenceDay – Recebe o valor do parâmetro (**pCtrlDivergencePeriod**) que é o total de divergências do período passado para controle de quando encerrar a correção dos dias.

VDivergenceDay - Recebe o valor da divergência das quantidades de linhas (**vContSelect** - **vCountLinesCsv**);

Função fCalcDivgCtrl = (\$1 - (\$2 - \$3)):

Esta função atualiza o valor da divergência a cada loop em que uma inconsistência é identificada e corrigida. Seu objetivo é reduzir gradualmente as divergências até que o valor seja zerado.

Parâmetros: (**vCtrlDivergenceDay**,**vContSelect**,**vCountLinesCsv**)

```
//Essa sub Verifica se contem divergências no dia a dia dos períodos. Após Localizar os dias divergente ela chama a correção do arquivo
SUB CorrDay(pSubFato,pFirstDayPeriod,pLastDayPeriod,pCtrlDivergencePeriod)
TRACE ENTREI NO DIA;
    LET vCtrlDivergenceDay = pCtrlDivergencePeriod;
    for vDateRef = pFirstDayPeriod to pLastDayPeriod
        vDateRef = Date(vDateRef)
        TRACE # $(vDateRef) #;

        CALL $(pSubFato)CountLinesSelect('TempCountSelect',vDateRef,vDateRef);
        CALL $(pSubFato)CountLinesCsv('TempTableCsv',vDateRef,vDateRef);

        //Pega o total de linhas que a tabela TempCount está armazenando, e coloca em uma variável
        LET vContSelectDay = Peek('COUNTLINESELECT', 0, 'TempCountSelect');

        //Pega o total de linhas que a tabela TempTable está armazenando, e coloca em uma variável
        LET vCountLinesCsvDay = NoOfRows('TempTableCsv');
        LET vDivergenceDay = (vContSelectDay - vCountLinesCsvDay);
        //Exclui as tabelas que estavam armazenando os totais de linhas
        CALL CleanTemps;

        vCtrlDivergenceDay = ${fCalcDivgCtrl(vCtrlDivergenceDay,vContSelectDay,vCountLinesCsvDay));

        Trace DIA -> Total = $(pCtrlDivergencePeriod). Controlador = $(vCtrlDivergenceDay). # ( $(vContSelectDay) - $(vCountLinesCsvDay) ) #;

        //Comparando os totais de linhas do arquivoQVD e do resultado da consulta no DW
        If vDivergenceDay <> 0 then
            CALL $(pSubFato)(vDateRef);
        end if;

        If vCtrlDivergenceDay = 0;
            Exit For;
        End If;
    next;
End SUB;
```

SubRotina CorrDay

For vDateRef = pLastDayPeriod to pFirstDayPeriod:

Este loop percorre os dias de um período passado, iniciando do último dia (pLastDayPeriod) até o primeiro (pFirstDayPeriod).

A análise é feita de trás para frente porque, logicamente, as divergências costumam ser encontradas nos últimos dias do mês.

vDateRef = Date(vDateRef) - Convertendo para data a variável que está percorrendo os dias do range(

IF vDivergenceDay <> 0 then

CALL \$(pSubFato)(vDateRef);

End If;

Este **IF** verifica se há divergências no dia analisado.

Se houver, a sub-rotina **\$(pSubFato)(vDateRef)** é chamada para corrigir a inconsistência, utilizando a data **vDateRef** como parâmetro.

If vCtrlDivergenceDay = 0;

Exit For;

End If;

Quando a variável que armazena o total de divergências (**vCtrlDivergenceDay**) atingir zero, indicando que todas as inconsistências foram corrigidas, o loop é interrompido.

```
//Essa sub Verifica se contem divergências no dia a dia dos períodos. Após Localizar os dias divergente ela chama a correção do arquivo
SUB CorrDay(pSubFato,pFirstDayPeriod,pLastDayPeriod,pCtrlDivergencePeriod)
TRACE ENTREI NO DIA;
    LET vCtrlDivergenceDay = pCtrlDivergencePeriod;
    for vDateRef = pFirstDayPeriod to pLastDayPeriod
        vDateRef = Date(vDateRef)
        TRACE # $(vDateRef) #;

        CALL $(pSubFato)CountLinesSelect('TempCountSelect',vDateRef,vDateRef);
        CALL $(pSubFato)CountLinesCsv('TempTableCsv',vDateRef,vDateRef);

        LET vDivergenceDay = (vContSelect - vCountLinesCsv);

        vCtrlDivergenceDay = $(fCalcDivgCtrl(vCtrlDivergenceDay,vContSelect,vCountLinesCsv));

        Trace DIA -> Total = $(pCtrlDivergencePeriod). Controlador = $(vCtrlDivergenceDay). # ( $(vContSelect) - $(vCountLinesCsv) ) #;

        //Comparando os totais de linhas do arquivoQVD e do resultado da consulta no DW
        If vDivergenceDay <> 0 then
            CALL $(pSubFato)(vDateRef);
        end if;

        If vCtrlDivergenceDay = 0;
            Exit For;
        End If;
    next;
End SUB;
```